10. 메모리 최적화 전략 분석

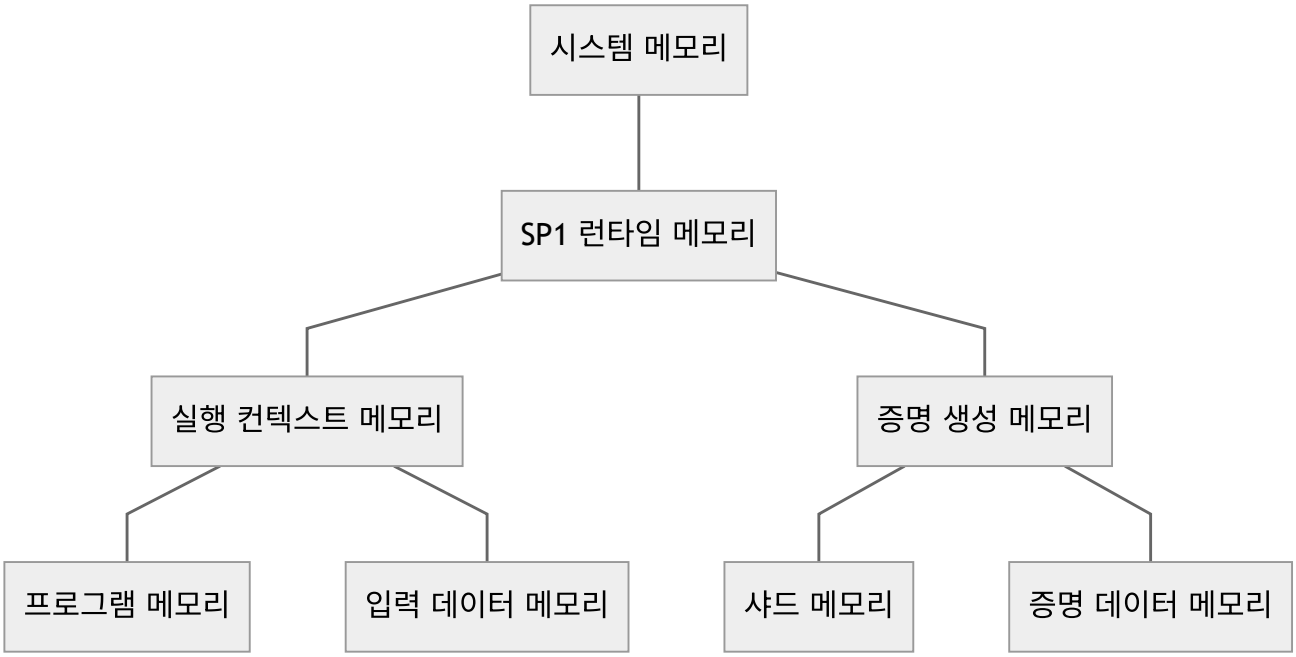
개요

SP1은 대규모 프로그램의 증명 생성을 위해 다양한 메모리 최적화 전략을 사용합니다. 이 문서는 SP1의 메모리 관리 방식, 효율적인 데이터 구조 사용법, 그리고 대용량 프로그램 처리를 위한 최적화 기법을 상세히 분석합니다.

메모리 관리 아키텍처

계층적 메모리 구조

SP1은 다음과 같은 계층적 메모리 구조를 사용합니다:



메모리 할당자 시스템

SP1은 실행 환경에 따라 다른 메모리 할당자를 사용합니다:

1. Embedded 할당자

```
// crates/zkvm/entrypoint/src/allocators/embedded.rs
pub static INNER_HEAP: Heap = Heap::empty();

pub fn init() {
extern "C" {
static _end: u8;
}
let heap_pos: usize = unsafe { (&_end) as *const u8 as usize };
assert!(heap_pos <= EMBEDDED_RESERVED_INPUT_START);
// 힙 크기 계산: 전체 메모리 - 예약된 입력 영역 - 힙 위치
let heap_size: usize = EMBEDDED_RESERVED_INPUT_START - heap_pos;
unsafe { INNER_HEAP.init(heap_pos, heap_size) };
}
```

특징:

- **예약된 입력 영역**: 1GB 크기의 전용 입력 데이터 영역
- **힙 분리**: 프로그램 힙과 입력 데이터 영역 분리
- **메모리 보호**: 입력 영역 메모리 해제 방지

2. Bump 할당자

```
// 단순한 범프 할당자로 빠른 할당/해제
unsafe extern "C" fn sys_alloc_aligned(bytes: usize, align: usize) -> *mut u
```

```
extern "C" {
static _end: u8;
}
let heap_pos = unsafe { (&_end) as *const u8 as usize };
let aligned_pos = (heap_pos + align - 1) & !(align - 1);
if aligned_pos + bytes > MAX_MEMORY {
panic!("Memory limit exceeded (0x78000000)");
}
aligned_pos as *mut u8
}
```

효율적인 데이터 구조 사용

1. HashMap vs BTreeMap 선택 전략

SP1은 용도에 따라 적절한 맵 구조를 선택합니다:

HashMap 사용 사례

```
// 빈번한 조회가 필요한 경우
use hashbrown::HashMap;

pub struct ShardProof<SC: StarkGenericConfig> {
pub chip_ordering: HashMap<String, usize>, // 칩 순서 매핑
pub public_values: Vec<Val<SC>>,
```

```
}

// 성능 최적화를 위한 hashbrown 사용
pub struct Shape<K: Clone + Eq + Hash> {
pub inner: HashMap<K, usize>, // O(1) 평균 조회 시간
}
```

BTreeMap 사용 사례

```
// 순서가 중요하거나 범위 쿼리가 필요한 경우
use alloc::collections::btree_map::BTreeMap;

pub struct Transcript {
pub challenges: BTreeMap<String, Challenge>, // 정렬된 챌린지 저장
previous_challenge: Option<Challenge>,
}

// 디버깅 및 분석용 정렬된 데이터
fn debug_interactions() -> (
BTreeMap<String, Vec<InteractionData<Val<SC>>>>,
BTreeMap<String, Val<SC>>
) {
// 정렬된 상호작용 데이터 반환
```

```
}
```

2. 벡터 최적화 전략

용량 사전 할당

```
// 메모리 재할당 최소화
let mut hash_input = Vec::with_capacity(48); // 정확한 크기 사전 할당

// 대용량 데이터 처리
let mut vec = Vec::with_capacity(D as usize); // D는 예상 크기
```

메모리 효율적인 읽기

```
pub fn read_vec() -> Vec<u8> {
let ReadVecResult { ptr, len, capacity } = unsafe { read_vec_raw() };
if ptr.is_null() {
return Vec::new();
}
// 기존 할당된 메모리 재사용
unsafe { Vec::from_raw_parts(ptr, len, capacity) }
}
```

3. 특화된 데이터 구조

Shape 클러스터링

```
pub struct ShapeCluster<K: Eq + Hash + FromStr> {
inner: HashMap<K, Vec<Option<usize>>>, // 계층적 형태 관리
}

impl<K> ShapeCluster<K> {
pub fn find_shape(&self, heights: &[(K, usize)]) -> Option<Shape<K>> {
// 최적의 형태 찾기 알고리즘
let shape: Option<HashMap<K, Option<usize>>> = heights
.iter()
.map(|(air, height)| {
// 메모리 효율적인 매핑
})
.collect();
}
}
```

샤드 기반 메모리 관리

샤드 크기 최적화

SP1은 시스템 메모리에 따라 동적으로 샤드 크기를 조정합니다:

```
// crates/stark/src/opts.rs
```

```
const MAX_SHARD_SIZE: usize = 1 << 21; // 2M 사이클

const RECURSION_MAX_SHARD_SIZE: usize = 1 << 22; // 4M 사이클


impl SP1ProverOpts {

pub fn cpu(cpu_ram_gb: usize) -> Self {

let (log2_shard_size, shard_batch_size, log2_divisor) =

Self::get_memory_opts(cpu_ram_gb);

let mut opts = SP1ProverOpts::default();

opts.core_opts.shard_size = 1 << log2_shard_size;

opts.core_opts.shard_batch_size = shard_batch_size;

opts

}

fn get_memory_opts(cpu_ram_gb: usize) -> (usize, usize, usize) {

match cpu_ram_gb {

0..=8 => (16, 1, 3), // 64K 사이클, 배치 크기 1

9..=16 => (18, 2, 2), // 256K 사이클, 배치 크기 2

17..=32 => (20, 4, 1), // 1M 사이클, 배치 크기 4

_ => (21, 8, 0), // 2M 사이클, 배치 크기 8

}

}

}
```

샤드 배치 처리

```
pub struct SP1CoreOpts {
pub shard_size: usize, // 개별 샤드 크기
pub shard_batch_size: usize, // 배치 처리 크기
}

// 메모리 사용량 = shard_size * shard_batch_size * 메모리_계수
```

메모리 풀링 및 캐싱 전략

1. 메모리 사용량 모니터링

SP1은 실시간 메모리 사용량을 모니터링합니다:

```
// crates/sdk/src/utils.rs
pub fn setup_memory_usage_monitoring() {
MEMORY_USAGE_MONITORING.call_once(|| {
spawn(|| {
let mut sys = System::new_with_specifics(
RefreshKind::new().with_memory(MemoryRefreshKind::new().with_ram()),
);
let total_memory = sys.total_memory();
loop {
sleep(Duration::from_secs(10));
sys.refresh_memory();
```

```
let used_memory = sys.used_memory();
let ratio = used_memory as f64 / total_memory as f64;
if ratio > 0.8 {
tracing::warn!(
"Memory usage is high: {:.2}%, we recommend using the prover network",
ratio * 100.0
);
}
}
});
});
}
```

2. 컨텍스트 기반 메모리 관리

```
pub struct SP1Context {
pub max_cycles: Option<u64>, // 사이클 수 제한
pub shard_size: usize, // 샤드 크기
pub checkpoints: Vec<u32>, // 체크포인트 위치
}

// 메모리 효율적인 설정
let context = SP1Context::builder()
```

```
.shard_size(1 << 20) // 1M 사이클 (작은 샤드)
.max_cycles(10_000_000) // 사이클 수 제한
.build();

// 성능 우선 설정
let context = SP1Context::builder()
.shard_size(1 << 24) // 16M 사이클 (큰 샤드)
.build();
```

3. 입력 데이터 캐싱

```
// 입력 데이터 전용 영역 관리
const EMBEDDED_RESERVED_INPUT_REGION_SIZE: usize = 1024 * 1024 * 1024; // 1GB
static mut EMBEDDED_RESERVED_INPUT_PTR: usize = EMBEDDED_RESERVED_INPUT_START;

fn read_vec_raw() -> ReadVecResult {
let len = syscall_read_len();
if len == usize::MAX {
return ReadVecResult { ptr: std::ptr::null_mut(), len: 0, capacity: 0 };
}
let capacity = (len + 3) / 4 * 4; // 4바이트 정렬
// 예약된 영역에서 메모리 할당
let ptr = unsafe { EMBEDDED_RESERVED_INPUT_PTR };
```

```
if ptr.saturating_add(capacity) > MAX_MEMORY {
panic!("Input region overflowed.");
}
unsafe { EMBEDDED_RESERVED_INPUT_PTR += capacity };
ReadVecResult { ptr: ptr as *mut u8, len, capacity }
}
```

대용량 프로그램 처리 최적화

1. 점진적 처리 전략

```
// 대용량 프로그램을 작은 단위로 분할 처리
impl Executor {
pub fn should_split_shard(&self) -> bool {
self.state.clk >= self.shard_size // 샤드 크기 초과 시 분할
}
pub fn check_cycle_limit(&self) -> Result<()> {
if let Some(max_cycles) = self.max_cycles {
if self.state.global_clk >= max_cycles {
return Err(ExecutionError::ExceededCycleLimit(max_cycles));
}
}
Ok(())
```

```
}

}
```

2. 스트리밍 처리

```
// 대용량 데이터의 스트리밍 처리

pub fn process_large_program(elf: &[u8], stdin: &SP1Stdin) -> Result<()> {

let context = SP1Context::builder()

.shard_size(1 << 20) // 작은 샤드로 메모리 사용량 제한

.max_cycles(u64::MAX) // 사이클 제한 없음

.build();

// 스트리밍 방식으로 처리

let mut executor = Executor::with_context(program, opts, context);

while !executor.is_finished() {

executor.execute_shard()?; // 샤드 단위로 처리

// 메모리 정리

if executor.should_cleanup_memory() {

executor.cleanup_intermediate_data();

}

}

Ok(())

}
```

3. 메모리 압축 기법

```
// 증명 데이터 압축
use bincode;

pub fn compress_proof_data(proof: &SP1CoreProof) -> Vec<u8> {
// 효율적인 직렬화로 메모리 사용량 감소
bincode::serialize(proof).expect("serialization failed")
}

pub fn decompress_proof_data(data: &[u8]) -> SP1CoreProof {
bincode::deserialize(data).expect("deserialization failed")
}
```

환경별 최적화 설정

1. 시스템 메모리 기반 자동 설정

```
impl Default for SP1CoreOpts {
fn default() -> Self {
let cpu_ram_gb = System::new_all().total_memory() / (1024 * 1024 * 1024);
let (default_log2_shard_size, default_shard_batch_size, default_log2_divisor) =
SP1ProverOpts::get_memory_opts(cpu_ram_gb as usize);
```

```
Self {
shard_size: env::var("SHARD_SIZE").map_or_else(
|_| 1 << default_log2_shard_size,
|s| s.parse::<usize>().unwrap_or(1 << default_log2_shard_size),
),
shard_batch_size: env::var("SHARD_BATCH_SIZE").map_or_else(
|_| default_shard_batch_size,
|s| s.parse::<usize>().unwrap_or(default_shard_batch_size),
),
// ... 기타 설정
}
}
}
```

2. GPU 메모리 최적화

```
impl SP1ProverOpts {
pub fn gpu(gpu_ram_gb: usize) -> Self {
let mut opts = SP1ProverOpts::default();
if 24 <= gpu_ram_gb {
let log2_shard_size = 21; // 2M 사이클
opts.core_opts.shard_size = 1 << log2_shard_size;
opts.core_opts.shard_batch_size = 1; // GPU는 배치 크기 1
```

```
} else {
// GPU 메모리 부족 시 더 작은 샤드 사용
opts.core_opts.shard_size = 1 << 20; // 1M 사이클
}
opts
}
}
```

성능 측정 및 튜닝

1. 메모리 사용량 프로파일링

```
// 메모리 사용량 측정
pub fn profile_memory_usage() {
let mut sys = System::new_all();
sys.refresh_memory();
println!("Total memory: {} GB", sys.total_memory() / (1024 * 1024 * 1024));
println!("Used memory: {} GB", sys.used_memory() / (1024 * 1024 * 1024));
println!("Available memory: {} GB", sys.available_memory() / (1024 * 1024 * 1024
}
```

2. 샤드 크기 벤치마킹

```
# 다양한 샤드 크기로 성능 테스트
declare -a shard_sizes=("262144" "524288" "1048576" "2097152" "4194304")

for shard_size in "${shard_sizes[@]}"; do
echo "Testing shard size: $shard_size"
SHARD_SIZE=$shard_size cargo run --release -- --program fibonacci
done
```

최적화 권장사항

1. 하드웨어별 설정

저사양 시스템 (8GB 이하)

```
let opts = SP1ProverOpts::cpu(8); // 자동으로 작은 샤드 크기 설정
// shard_size: 64K, batch_size: 1
```

중간 사양 시스템 (16GB)

```
let opts = SP1ProverOpts::cpu(16);
// shard_size: 256K, batch_size: 2
```

고사양 시스템 (32GB+)

```
let opts = SP1ProverOpts::cpu(32);
```

```
// shard_size: 1M, batch_size: 4
```

2. 프로그램 특성별 최적화

메모리 집약적 프로그램

```
let context = SP1Context::builder()
.shard_size(1 << 18) // 256K 사이클 (작은 샤드)
.max_cycles(1_000_000) // 사이클 제한
.build();
```

연산 집약적 프로그램

```
let context = SP1Context::builder()
.shard_size(1 << 22) // 4M 사이클 (큰 샤드)
.build();
```

3. 환경 변수 활용

```
# 샤드 크기 직접 설정
export SHARD_SIZE=1048576

# 배치 크기 설정
export SHARD_BATCH_SIZE=2
```

```
# 메모리 사용량 모니터링 활성화
export SP1_MEMORY_MONITOR=1
```

결론

SP1의 메모리 최적화 전략은 다층적 접근 방식을 사용합니다:

1. **적응적 할당자**: 실행 환경에 맞는 메모리 할당자 선택
2. **효율적인 데이터 구조**: 용도에 맞는 컬렉션 타입 사용
3. **샤드 기반 분할**: 메모리 사용량을 제어하는 샤드 시스템
4. **동적 최적화**: 시스템 리소스에 따른 자동 설정 조정
5. **실시간 모니터링**: 메모리 사용량 추적 및 경고

이러한 전략을 통해 SP1은 제한된 메모리 환경에서도 대규모 프로그램의 증명 생성을 효율적으로 수행할 수 있습니다.